# Week 25 (capstone applied lecture)
# Synthetic ATS RL Enhancement Audit

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

# Contents


**Learning objectives** 4

**1 Motivation and intuition** 5
- 1.1 What makes recruitment hard for RL . . . 5
- 1.2 Bird's-eye view of what follows . . . 5

**2 System tour: a synthetic ATS case study** 7
- 2.1 Stack and data flow . . . 7
- 2.2 Block diagram . . . 7
- 2.3 Where the implicit policies live . . . 8

**3 Mathematical formulation: decision-point catalog** 9
- 3.1 Why most of the catalog is a bandit, not a full MDP . . . 10
- 3.2 Where the bandit reduction breaks — and the right tool changes . . . 10
- 3.3 State featurisation . . . 11

**4 Algorithms: per-decision RL method mapping** 12
- 4.1 D1 in detail — the headline recommendation . . . 12
- 4.2 D5 in detail — DPO for outreach text . . . 13
- 4.3 D12 in detail — offline RL on salary anchors . . . 13

**5 Implementation walkthrough** 15

**6 Empirical results** 17
- 6.1 Reading the OPE plot . . . 17

**7 Reward design and the delayed-feedback gauntlet** 20
- 7.1 Hire as the canonical reward — and its three failure modes . . . 20
- 7.2 Goodhart for the LLM components (D5, D11) . . . 20
- 7.3 Fairness quantities and feedback . . . 20
- 7.4 Cost-aware reward (D2) . . . 21
- 7.5 Counterfactual hire signal — the synthetic insurance parallel . . . 21

**8 Off-policy evaluation plan** 22
- 8.1 What the existing event log records . . . 22
- 8.2 What the log is missing for OPE . . . 22
- 8.3 Estimator stack and uncertainty . . . 22
- 8.4 Data quality checks . . . 23

**9 Safety, fairness, and compliance** 24

**10 Phased rollout roadmap** 25

**11 Empirical demo summary** 27

**12 Matching application** 28

**13 Exercises** 29

**14 Further reading** **30**

# Learning objectives

**Study companion.** Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

**Key Idea**

By the end of this lecture you will be able to:

1. Map the synthetic ATS pipeline end-to-end and identify *every* decision point where a policy is implicit today (heuristic, supervised scorer, LLM agent committee).
2. For each decision point, match a candidate RL method from weeks 1–24 by problem geometry — bandit vs sequential, online vs offline, constrained vs unconstrained, scalar vs preference.
3. Design Goodhart-resistant rewards with explicit fairness as a constraint (not an objective), and propose three reward hacks per decision point you can pre-empt.
4. Sketch an Off-Policy Evaluation plan that the existing event log could support, and the minimum schema change required if it does not.
5. Argue the EU AI Act high-risk classification implications for each proposed RL component, with concrete article references and human-in-the-loop checkpoints.
6. Lay out a six-month phased rollout: from instrumented logging to shadow-mode bandit to constrained online learning to LLM DPO fine-tuning, with empirical go/no-go gates.
7. Reproduce a Phase-1 demo end-to-end (LinTS reranker on top of a heuristic scorer, OPE-validated, fairness-audited).

# 1 Motivation and intuition

This is the capstone week. The previous 24 weeks built up the RL toolbox: bandits, dynamic programming, value-based methods, policy gradients, offline RL, sequence modelling, LLM alignment, safe and constrained RL. This week we point the toolbox at a hypothetical recruitment platform and ask, for every place the proposed system makes a decision, *what is the strongest RL-flavoured replacement we could justify, and what would it cost to ship?*

The audit is a design exercise supported by the local synthetic simulator. We define decision points, document assumptions, and propose evaluations. The deliverable is a defensible experimental roadmap.

*Intuition.* Even a rule-based system implicitly runs a policy. A scoring committee, a heuristic scorer, any blending coefficients, and the shortlist size ($k = 10$) — these are all hand-tuned hyperparameters of a fixed policy. Learning may improve a specified objective when feedback and modelling assumptions support it. Statistical guarantees, fairness definitions and an audit trail are separate requirements; adding an RL algorithm does not supply them automatically.

## 1.1 What makes recruitment hard for RL

Three structural features dominate the design space and recur in every chapter of this lecture:

1. **Delayed, partial, censored feedback.** A hire is the ground truth label, but it arrives weeks or months after the re-rank decision. Most candidates are filtered out before the feedback signal is recorded at all. Most ATS event logs see only clicks, opens, shortlist promotions — which are leading indicators but actively misleading proxies (Goodhart Week 21).

2. **High-stakes, regulated, audited.** Recruitment is explicitly classified as a high-risk AI system under the EU AI Act (Annex III, point 4). This pushes the design squarely into constrained RL (week 22) plus shielding (week 23) plus comprehensive logging.

3. **LLM-in-the-loop everywhere.** The agent committee, the outreach drafter, the interview question generator are all LLM-driven. Their behaviour is a policy in policy-space; the natural tools are DPO/GRPO/RLHF (weeks 18–20), not classical value-based RL.

**Common pitfall**

A naive read of the ATS architecture is "it's a supervised classification problem on top of LLM features." This is wrong twice. First, the *logged labels* are selection outcomes from past policies, so they are systematically biased by the policy that generated them (week 11 lesson). Second, the system is sequential: today's reranking shapes tomorrow's candidate pool through outreach, response rates, and self-selection. Both of these push us out of supervised land.

## 1.2 Bird's-eye view of what follows

Chapter 2 tours the system, with a TikZ block diagram. Chapter 3 is the **decision-point catalog**, the core artefact of the audit — a numbered table that the rest of the document drills down into.

Chapter 4 maps each decision point to one or more RL methods from weeks 1–24, justified by problem geometry. Chapter 5 designs Goodhart-resistant rewards. Chapter 6 spells out the OPE plan. Chapter 7 is the long safety chapter (longest of any week — spec requirement). Chapter 8 is the six-month phased rollout. Chapter 9 is the empirical demo from `project/`. Chapter 10 closes with the matching callout and exercises.

# 2 System tour: a synthetic ATS case study

This chapter defines an illustrative applicant-tracking system for the exercises. Its components are design assumptions, not a description of an external company's implementation. The runnable simulator is entirely contained in this week's `project/` directory.

**Common pitfall**

**Evidence boundary.** This study repository contains synthetic project code and locally generated data. The architecture below is a hypothetical teaching design. Toy metrics, roadmap dates, and passing exercise checks do not validate a live system.

## 2.1 Stack and data flow

- Interface: a candidate-upload and recruiter-review application.
- Backend: a service coordinating parsing, scoring, and ranking.
- Storage: transactional records, an embedding index, and an audit log.
- Model runtime: interchangeable scoring models with an explicit cost and latency budget.
- Document parser: text extraction with optional OCR for scanned PDFs.
- Interview interface: optional transcription and structured feedback.

The candidate lifecycle traverses: *ingest* (CV upload, OCR, parse) → *evaluate* (multi-agent committee, scoring) → *rank* (rubric engine, blended score) → *outreach* (email templates, calendar) → *interview* (voice or in-person, with the voice interface) → *decide* (recruiter action, sometimes second-round) → *hire / reject*.

## 2.2 Block diagram

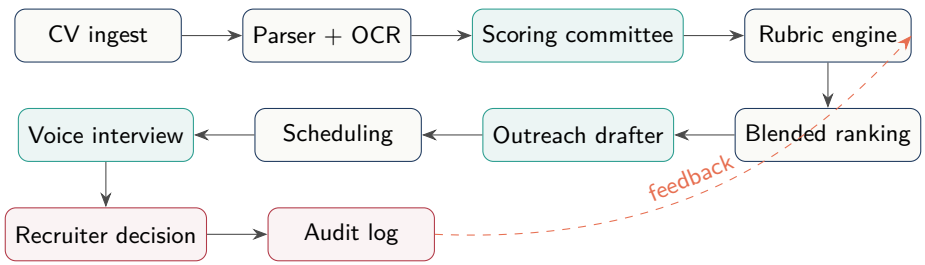



Figure 2.1: The synthetic ATS pipeline. Blue nodes are deterministic modules. Teal nodes are LLM-driven. Red nodes are human-in-the-loop or audit-critical. The dashed orange edge is the (currently *missing* or under-exploited) feedback loop that any RL component would consume.

## 2.3 Where the implicit policies live

This hypothetical design has seven illustrative policy decision points:

1. **Scoring weights**: a hand-chosen weighted sum of skill similarity and experience fit. The local simulator uses a 70/30 mixture in `project/scorer.py`; these are teaching constants.
2. **Blending coefficient**: how to combine model scores with requirements coverage, if that additional signal is introduced.
3. **Mismatch penalty**: how strongly to penalise a candidate-job mismatch, under an explicitly defined distance measure.
4. **Model routing**: choosing a scoring model under a cost-quality budget, an illustrative contextual-bandit problem.
5. **Red-flag gate** (PASS/FAIL) — binary policy with a fixed threshold.
6. **Outreach copy + send-time** — templates today, prime candidates for DPO + a small MDP.
7. **Interview question selection** — chooses follow-up questions from CV gaps; classic contextual bandit.

**Common pitfall**

The implicit-policy view is uncomfortable for system owners who think of their codebase as "rules, not models". The cleaner reframe: every hand-tuned hyperparameter is a model with a sample size of one (the engineer's intuition). RL is the discipline of replacing that sample of one with a controlled experiment.

# 3 Mathematical formulation: decision-point catalog

We catalogue every RL-candidate decision point and formalise its state / action / reward / feedback delay / safety class. The numbering is used throughout the rest of the document.

| # | Decision | State $s$ | Action $a$ | Reward $r$ | Delay |
|---|---|---|---|---|---|
| D1 | Candidate re-rank within job's shortlist | cand+job feature vec, slate context | pick position in slate of size $k$ | hire $\in \{0, 1\}$ | weeks |
| D2 | Velocity-tier routing of LLM call | job class, agent type, queue depth | {fast, balanced, deep, premium} | quality $-\ \lambda \cdot$ cost | seconds |
| D3 | Agent weight assignment per job archetype | job archetype, role family | weight simplex $w \in \Delta^5$ | calibrated hire probability | weeks |
| D4 | Red-flag threshold | redflag_score | accept / reject / human-review | fraud true positive rate − false-positive cost | days |
| D5 | Outreach copy (LLM-drafted) | cand+job context, channel | generated text | response / accept rate | hours–days |
| D6 | Outreach send-time | cand timezone, past open history | timestamp | open / reply rate | hours |
| D7 | Interview challenge question selection | CV gap set, archetype | one of $K$ probe questions | information gain | minutes |
| D8 | Interviewer-pool assignment | cand profile, interviewer skills/load | one of $m$ recruiters | interview-quality − idle-time | days |
| D9 | Slot offer in calendar negotiation | calendar state, candidate availability hist. | set of offered slots | acceptance rate − no-show rate | hours–days |
| D10 | Re-evaluation trigger after new signal | evaluation row + new evidence | rerun {None, Phase1, Full} | info gain − compute cost | seconds |
| D11 | Job description language polish | raw JD, target locale | generated text | applicant quality / applicant volume | days |
| D12 | Salary-anchor in offer letter | cand seniority, market band | offer amount | acceptance · retention-12mo | months |

| | | | | | |
|---|---|---|---|---|---|
| D13 | Career-trajectory next-step forecast | cand history sequence | predicted next role / signal | predictive log-likelihood | months–years |

**Definition 3.0.1** (Recruiter Markov Decision Process (R-MDP))**.** A finite-horizon MDP $(S, A, P, R, \gamma)$ where: $S$ is the joint candidate-job-context space, $A$ contains all of D1–D13, $P$ is the (unknown) transition operator induced by candidate response, recruiter intervention and time decay, $R : S \times A \to \mathbb{R}$ is a scalar objective. Fairness, compliance and resource budgets require separately defined cost channels rather than an undeclared vector reward, $\gamma$ is a calendar-time discount (months to a hire).

The hypothetical system can be modeled as an R-MDP. The audit's job is to enumerate which components of the R-MDP can be brought under explicit, statistical control.

## 3.1 Why most of the catalog is a bandit, not a full MDP

For most of D1–D11 the next state is essentially independent of the action: re-ranking candidate $i$ today does not change which candidate $i + 1$ shows up tomorrow (modulo the bias the policy injects into who gets seen, addressed in chapter 6). Treating each decision as a one-step contextual bandit is a modelling approximation when delayed effects and shared-resource coupling can be neglected. Under those assumptions you can use the OPE machinery of week 11 when support and logging assumptions hold; regret bounds require additional model assumptions.

Three exceptions:

- D6 (send-time) has temporal correlation across messages to the same candidate. Small MDP.
- D9 (slot offer) has calendar-state evolution. Small MDP.
- D12 (salary anchor) propagates into retention. Full long-horizon MDP — and the place offline RL (week 14) is most defensible.

**Theorem 3.1.1** (Bandit reduction)**.** *If $P(s_{t+1}|s_t, a_t) = P(s_{t+1}|s_t)$ for all $a_t$ (no action-induced transitions) and the reward depends only on $(s_t, a_t)$, then the optimal policy for the R-MDP at every state $s$ is given by the greedy bandit policy $\pi^\star(s) = \arg\max_a \mathbb{E}[r|s, a]$, for an unconstrained additive discounted objective. LinUCB/LinTS regret bounds additionally require their own feature, noise, action-set and model assumptions.*

*Proof sketch.* With actions decoupled from transitions the Bellman equation collapses to $V^\star(s) = \gamma\mathbb{E}_{s'}[V^\star(s')] + \max_a \mathbb{E}[r|s, a]$. Iterating gives $V^\star(s) = \sum_{t\geq 0} \gamma^t \mathbb{E}[\max_a \mathbb{E}[r|s_t, a]]$ — a discounted sum of statewise optimal immediate values. States may remain temporally correlated; they need not be independent. Coupled capacity or fairness constraints also invalidate a purely greedy reduction. □ □

## 3.2 Where the bandit reduction breaks — and the right tool changes

For D9 and D12, the action shapes the state. For D9 in particular, an unaccepted slot offer permanently consumes calendar inventory; for D12 the offer amount shapes downstream retention. Neither admits the bandit reduction. They need at minimum:

- D9: a small MDP solvable with PPO (week 9), and a *shield* (week 23) preventing impossible offers.
- D12: offline RL (IQL / CQL, week 14) on the historical offer-acceptance-retention dataset, with conservatism on out-of-distribution offer amounts.

## 3.3 State featurisation

For every decision point in the catalog the state vector should include the *static* candidate and job features — skill vectors, years, seniority, archetype — plus the *historical* signals the existing event log would need to record: number of previous evaluations of this candidate, disagreement between scoring models, composite confidence, and any red-flag score.

The composite confidence is particularly interesting: it is already an implicit *uncertainty estimate*, which is exactly the input a Thompson-sampling reranker would want. The audit recommends exposing it unmodified to the bandit.

# 4 Algorithms: per-decision RL method mapping

For each catalogued decision we propose one to three RL methods, pulled from the curriculum.

| # | Primary method | Backup / pretraining | Justification |
|---|---|---|---|
| D1 | LinTS reranker (wk 11) | NeuralUCB (wk 11 extension) | Pure contextual bandit, fast, defensible. |
| D2 | Contextual bandit with cost (wk 11) | RL-VLM router | Action set small ($\leq$ 4), reward = quality $-$ $\lambda$·cost. |
| D3 | Bayesian linear regression $\rightarrow$ weight posterior | Meta-bandit (wk 17) per archetype | Few observations per archetype; need shrinkage. |
| D4 | CMDP shield + bandit (wks 22, 23) | — | Hard compliance constraint on false rejects. |
| D5 | DPO (wk 19) on logged accept/reject preferences | RLHF + PPO (wk 18) | Pairwise preferences; DPO removes a separate reward model but still inherits proxy and preference-data failure modes. |
| D6 | PPO over send-time MDP (wk 9) | Multi-armed bandit if i.i.d. | Cyclic features (hour, day-of-week). |
| D7 | LinUCB on probe questions (wk 11) | Decision Transformer (wk 15) on interview transcripts | Pure information-gain bandit. |
| D8 | Constrained bipartite matching (wk 12) | — | Load balancing constraint; not really RL but lives in this family. |
| D9 | PPO-Lagrangian (wk 22) | Shield (wk 23) for impossible offers | Calendar state evolves; safety constraints are hard. |
| D10 | Threshold bandit (wk 11) | — | Trivial: scalar threshold, scalar reward. |
| D11 | GRPO (wk 20) with verifier on applicant quality | DPO (wk 19) | Group-relative makes for cheap training; verifiable signal. |
| D12 | IQL or CQL (wk 14) offline; never online | DT (wk 15) for trajectory features | Cannot explore online; conservatism essential. |
| D13 | Decision Transformer (wk 15) | — | Sequence modelling problem. |

## 4.1 D1 in detail — the headline recommendation

D1 (candidate re-ranking within a job's shortlist) is the highest-value, lowest-risk decision point and where the audit recommends starting. The pipeline is:

1. Existing rubric engine produces a top-$k$ shortlist (we keep $k = 10$ today; this is a tunable per-job hyperparameter).
2. The shortlist features are passed to the LinTS reranker.
3. LinTS draws $\tilde{\theta}_t \sim \mathcal{N}(\hat{\theta}_t, v^2 A_t^{-1})$ and orders the slate by $\tilde{\theta}_t^\top \phi(c, j)$.

4. The top reranked candidate is surfaced first to the recruiter. The logged action is the randomised display choice. The subsequent recruiter decision and eventual outcome are separate events; the human's conditional action probability is not automatically known.

**Algorithm 1:** LinTS reranker on top of the synthetic ATS rubric engine.

**Input:** Feature map $\phi$, prior precision $\lambda$, variance scale $v^2$.
$A \leftarrow \lambda I_d,\ b \leftarrow 0$
**for** *each requisition* $(j, slate_t)$ *arriving online* **do**
  $\hat{\theta} \leftarrow A^{-1}b$ `// posterior mean`
  $\tilde{\theta} \sim \mathcal{N}(\hat{\theta}, v^2 A^{-1})$ `// posterior sample`
  rank slate by $\tilde{\theta}^\top \phi(c, j)$
  randomised display chooses $c^\star$ with known propensity; log that action and probability
  observe delayed reward $r \in \{0, 1\}$ when the requisition closes
  $A \leftarrow A + \phi(c^\star, j)\phi(c^\star, j)^\top$
  $b \leftarrow b + r\,\phi(c^\star, j)$

The rank-one precision update costs $O(d^2)$ and scoring $k$ candidates costs $O(kd)$. A fresh matrix solve or covariance factorisation for Gaussian sampling costs $O(d^3)$ without maintained inverse/factor updates. Treat these separately when estimating runtime.

**Example 4.1.1** (Worked numerical example)**.** Take $d = 4$, prior precision $I$, $v^2 = 1$, feature vectors $(1, 0, 0, 0)$, $(0, 1, 0, 0)$, $(1, 1, 0, 0)$, and rewards $(1, 0, 1)$. Then $b = (2, 1, 0, 0)^\top$ and

$$A = \begin{pmatrix} 3 & 1 & 0 & 0 \\ 1 & 3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad \hat{\theta} = A^{-1}b = (5/8, 1/8, 0, 0)^\top.$$

The unseen coordinates retain their prior variance 1, even though their posterior means are zero. Exploration depends on uncertainty in the sample, not merely on mean shrinkage. A Gaussian linear model is a teaching approximation for binary hire outcomes; it does not make its posterior uncertainty automatically calibrated.

## 4.2 D5 in detail — DPO for outreach text

For D5 (outreach copy generation) we follow the DPO recipe from week 19. The data pipeline is:

- Recruiter is offered two LLM-drafted variants of the same outreach email at compose time.
- Recruiter picks one (or edits one; either signal counts as "chosen").
- Pairs $(x, y_w, y_l)$ accumulate over weeks.
- DPO loss $\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}\big[\log \sigma\big(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\big)\big]$ on the open-source small LLM that drafts the outreach.

The key technical risk is length bias (week 19 / 21): DPO can prefer longer responses regardless of quality. Mitigation is length-controlled DPO — penalise the score by token count differential.

## 4.3 D12 in detail — offline RL on salary anchors

D12 is the only decision in the catalog where we recommend *no online learning, ever*. The action (offer amount) influences retention on a 6–24 month horizon, and counterfactuals are not observed

(offers are made once). The right tool is offline RL on the historical offer → acceptance → tenure dataset, with IQL (Implicit Q-Learning, week 14):

$$\mathcal{L}_V = \mathbb{E}[L_2^\tau(Q(s,a) - V(s))], \quad \mathcal{L}_Q = \mathbb{E}[(r + \gamma V(s') - Q(s,a))^2]$$

with $\tau$ the expectile (week 14 derivation). IQL avoids the out-of-distribution action problem of vanilla Q-learning on offline data; conservatism is the whole point.

# 5 Implementation walkthrough

We implemented the Phase-1 demo in `project/`. The files mirror the catalog:

- `env.py` — synthetic candidate × job population with a sensitive attribute that is correlated with years-of-experience (the realistic confound).
- `scorer.py` — the baseline heuristic scorer mimicking the rubric engine's blended score.
- `bandit.py` — LinTS reranker.
- `ope.py` — SNIPS, Doubly Robust, ESS.
- `constrained.py` — a small PPO-Lagrangian-style trainer that illustrates a chosen synthetic selection-share target.
- `audit.py` — selection rates, calibration buckets, equalised-odds gap.
- `shield.py` — a minimal hard-policy shield.
- `cli.py` — single candidate-job match endpoint.

```python
# project/bandit.py - Linear Thompson Sampling reranker.
class LinTSReranker:
    def __init__(self, d, v2=1.0, lam=1.0, seed=0):
        self.A = lam * np.eye(d)
        self.b = np.zeros(d)
        self.v2 = v2
        self.rng = np.random.default_rng(seed)
    def sample_theta(self):
        return self.rng.multivariate_normal(
            np.linalg.solve(self.A, self.b),
            self.v2 * np.linalg.inv(self.A))
    def score(self, X):  # X is (k, d) slate
        return X @ self.sample_theta()
    def update(self, x, r):
        self.A += np.outer(x, x); self.b += r * x
```

The OPE module follows the week-11 conventions: SNIPS as primary estimator, DR as the bias-corrected option when a reward model is available, bootstrap CIs to communicate uncertainty.

```
# project/ope.py - Self-Normalised IPS.
def snips(log, pi):
    num = den = 0.0
    for x, a, r, pb, slate in log:
        if not 0 < pb <= 1:
            raise ValueError("Invalid logged propensity")
        w = pi(slate)[a] / pb
        num += w * r; den += w
    if den <= 0:
        raise ValueError("No observed target-policy support")
    return num / den
```

The constrained module implements a single-step Lagrangian on top of a linear reward policy. It is small on purpose — the week 22 lecture has the full PPO-Lagrangian treatment. Here we just want to demonstrate the constraint dynamics.

**Common pitfall**

A fixed penalty coefficient does not generally enforce a chosen budget. An adaptive Lagrange multiplier can seek the constrained optimum under feasibility and optimisation assumptions, but expectation constraints do not imply zero violations in every decision or batch. The toy `constrained.py` uses a deterministic score adjustment, a reward-weighted parameter update, and a nondecreasing multiplier driven by a clipped shortfall. It demonstrates feedback mechanics; it is not PPO or an exact primal-dual solver. Its group statistics are shares among selected candidates, not selection rates conditional on group eligibility.

# 6 Empirical results

We ran the Phase-1 demo on 400 candidates × 60 jobs, $T = 1500$ logged decisions, $k = 10$ shortlist, $\varepsilon = 0.05$ uniform mass to guarantee overlap. Results below are from `project/make_figures.py` (deterministic, seed 0).

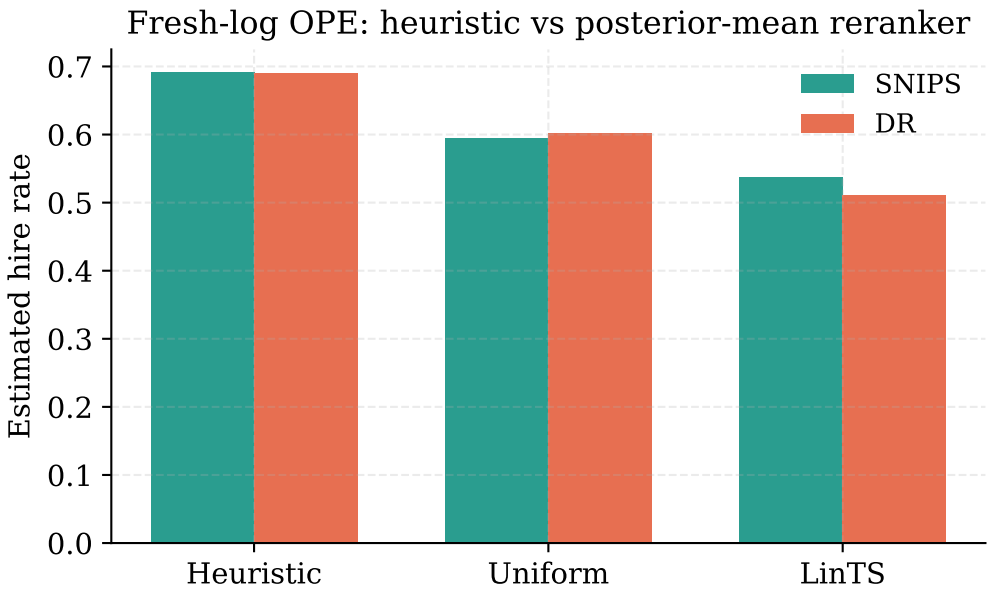



Figure 6.1: Fresh-log off-policy estimates for three fixed target policies. The LinTS-labelled target exploits a frozen posterior mean, rather than resampling a Thompson parameter per estimator call. Estimator differences need interpretation alongside overlap and the synthetic expected values.

## 6.1 Reading the OPE plot

The driver fits on one logged stream and evaluates on another, under the same synthetic population and shortlist rule. It also reports expected values from the known simulator to separate estimation noise from actual policy performance. Read the comparison with these caveats:

1. SNIPS is generally biased in finite samples, with no fixed direction. With $T = 1500$ and $k = 10$ the effective sample size on extreme target policies (like LinTS-argmax) can be small; we report bootstrap CIs in the workbook E4 to surface this.

2. DR has an exact population correction when either nuisance component is correctly specified under support and sampling assumptions; consistency of estimated components concerns asymptotics. Correct logged propensities can support the DR correction even with a misspecified reward model; misspecification alone does not establish pessimism. The implementation now freezes its posterior-mean target and uses fresh evaluation logs. This resolves fitting reuse and callback randomness; it does not resolve weak overlap or extend support beyond the heuristic shortlist.

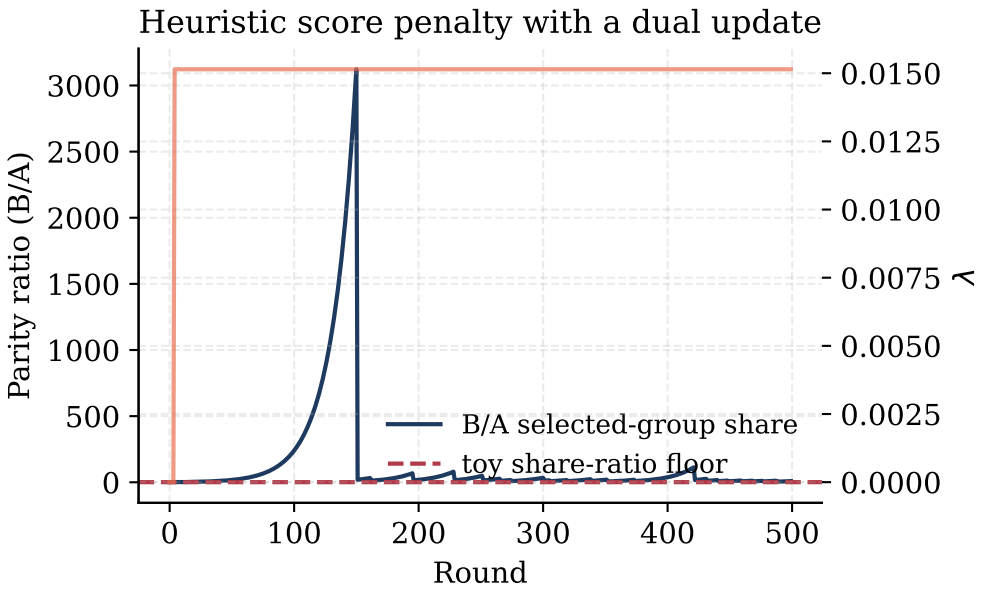



Figure 6.2: Heuristic score penalty with a dual update. The multiplier grows when group B's selected share falls short of the toy ratio floor. Selected shares lack applicant exposure denominators, so this is not a four-fifths hiring-rate audit or a PPO implementation.

The take-away mirrors week 11: never report a single OPE number, always report both SNIPS and DR, and bracket with bootstrap CIs.

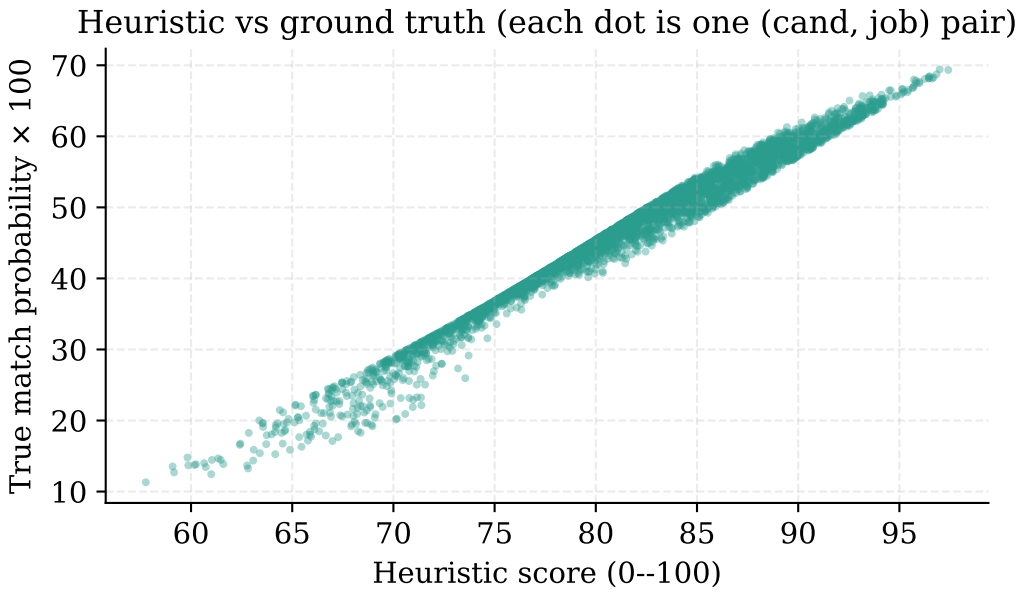



Figure 6.3: Heuristic score versus synthetic match probability. A rubric score is not an outcome-probability prediction; departures from the diagonal are not by themselves a probability-calibration failure.

# 7 Reward design and the delayed-feedback gauntlet

Reward design is a central failure mode examined by this synthetic audit. We work through each catalog item.

## 7.1 Hire as the canonical reward — and its three failure modes

The natural reward for D1, D3, D4, D5, D6, D8, D9, D11 is the binary hire indicator. Three failure modes are immediate:

1. **Survivorship bias.** Only candidates who are surfaced get a hire signal. Filtered candidates are silently treated as zero-reward, which they are not — they are *unobserved*. Mitigation: log a propensity at the shortlist stage; treat non-shortlisted candidates as missing-at-random conditional on the shortlist policy (week 11 framing).

2. **Time censoring.** A candidate hired in month 3 is a positive; one still in the funnel at month 3 is right-censored. Treating censored examples as zero reward biases the estimate downward and accelerates abandonment of slow funnels. Mitigation: survival-analysis-style discounting, or use a leading-indicator reward and validate it against the eventual hire offline.

3. **Counterfactual hire.** Sometimes a recruiter overrides the reranking and hires someone the policy ranked low. This is *evidence about the recruiter*, not the policy. Mitigation: log the recruiter override as a separate signal; learn an auxiliary recruiter model and condition the policy on it.

## 7.2 Goodhart for the LLM components (D5, D11)

For the LLM-driven decisions, the most common reward hack is length bias plus sycophancy:

- Long outreach emails get more replies — not because they are better but because recipients feel obligated to respond. Reward per-token is therefore mis-calibrated.

- LLM job-description polishers can drift toward generic phrasing that scores well on every applicant-quality classifier but actually reduces applicant volume of the right type.

The DPO mitigations are length-controlled DPO and verifiable rewards (RLVR-style, week 20) where possible — e.g. a verifier that penalises any phrasing flagged by a legal compliance regex.

## 7.3 Fairness quantities and feedback

A measurable constraint requires a denominator, population, time window and budget. A fixed reward penalty $J_r - \lambda J_c$ can recover a constrained optimum at a suitable optimal multiplier under duality assumptions, but an arbitrary chosen $\lambda$ does not specify a particular cost budget. Adaptive multipliers help search for that tradeoff; they do not solve an infeasible constraint or make a noisy proxy legally sufficient. Detect infeasibility and numerical saturation rather than letting the multiplier rise forever.

The shipped constrained demonstration reports shares among selected candidates. Those shares are not the group-conditional selection rates used for demographic parity. Report eligible group counts before interpreting a selection-rate ratio. A finite-batch feedback trace is not proof of population feasibility.

## 7.4 Cost-aware reward (D2)

For velocity-tier routing the reward is naturally cost-aware: $r = \text{quality}(a) - \lambda \cdot \text{token_cost}(a)$. The trick is that quality is itself a noisy estimator from a recruiter-rating loop. We recommend a two-tier reward: an immediate cost signal (cents per call) plus a delayed quality signal (recruiter rating, $\leq$ 10% sampled). The optimal $\lambda$ comes from the business — it is the dollars-per-quality-point the platform is willing to spend.

## 7.5 Counterfactual hire signal — the synthetic insurance parallel

The synthetic insurance lead-routing example has selective feedback: only the routed broker's conversion is observed. A matching experiment similarly needs an explicit logged action, known randomisation probability and aligned outcome. The propensity of a displayed candidate is not the probability of a recruiter's later choice. Human overrides, missing outcomes and delayed labels require separate modelling; assigning human choices probability one does not establish ignorability or support.

# 8 Off-policy evaluation plan

Every proposed RL component must be OPE-validated before any live deployment. We sketch the plan.

## 8.1 What the existing event log records

For this hypothetical architecture, a useful starting event schema would include the following fields. The list is a design exercise; it does not describe any external API or database:

- Evaluation id, candidate id, job id, company id, timestamps.
- Per-agent scores and composite confidence.
- Scoring output: overall score, uncertainty, and decision flags.
- Recruiter actions: shortlist promotion, reject, interview.
- Hire / no-hire outcomes (when recorded; often missing).

## 8.2 What the log is missing for OPE

Record the decision-time context and eligible action set, policy version, chosen action, its actual conditional probability, timestamp, and outcome availability. A deterministic policy has zero probability on alternatives; without additional outcome-model assumptions, their values are not identified from its log. A human policy's probabilities cannot be inferred from a single observed choice.

For a synthetic $\varepsilon$-mixture over $k$ eligible actions,

$$p_B(a|x) = (1-\varepsilon)\mathbb{1}[a = a_0(x)] + \varepsilon/k.$$

This only supplies support inside that eligible set. With $k = 10$ and $\varepsilon = 0.05$, its minimum probability is 0.005 and a deterministic target's importance weight can reach 200, not 20.

## 8.3 Estimator stack and uncertainty

Use IPS, SNIPS and a cross-fitted doubly robust estimate together with weight histograms and effective sample size. SNIPS is generally biased in finite samples; self-normalisation does not universally lower variance. The DR identity is exact with correct propensities or a correct conditional reward model under support and the appropriate independence assumptions. Consistency of estimated nuisance models is an asymptotic statement, not finite-sample unbiasedness. Cross-fitting helps separate evaluation rows from reward-model fitting.

For independent observations and finite second moments, the ratio-estimator approximation is

$$\mathrm{Var}(\widehat{V}_{\mathrm{SNIPS}}) \approx \frac{\mathrm{Var}_B(w(R-V))}{n(\mathbb{E}_B w)^2}.$$

The covariance of weights and rewards matters. A single worst-case weight cannot determine a universal sample size. Use a power calculation for the chosen estimand and dependence structure; inspect bootstrap stability and zero-weight resamples. Cluster related decisions when their outcomes are not independent. ESS is a diagnostic, not a guarantee that a confidence interval has nominal coverage.

## 8.4 Data quality checks

Verify target support over the entire eligible action set, reconstruct features at decision time, and record actual event-specific propensities. A changing logger can still support unbiased IPS when each propensity is correct. Population drift changes the target estimand and may require a separate transport argument; a universal KL threshold does not establish validity. Missing hire outcomes need a justified censoring model, rather than silently becoming zero rewards.

# 9 Safety, fairness, and compliance

**Safety angle**

**Scope and dated sources (checked 20 October 2026).** This is a synthetic teaching audit, not a legal compliance determination. Employment uses can be high-risk under Article 6 and Annex III of the EU AI Act; classification depends on intended purpose and applicable exceptions. In the Act's framework, technical documentation and logs are Articles 11 and 12; transparency, oversight and robustness are Articles 13–15. The Commission's current overview lists 2 December 2027 for Annex III high-risk obligations following the AI Omnibus amendments. Check the current legal text for any real system's role, scope and transitional rules: European Commission AI Act overview.

**Human review is substantive.** A nominal human click does not settle GDPR Article 22. Meaningful intervention needs an empowered reviewer who can assess and change the decision. Solely automated significant decisions, exceptions and safeguards need context-specific analysis; reranking is not a blanket exemption. See the EDPB-endorsed automated-decision guidelines.

**Measure the right denominator.** The US four-fifths comparison uses selection rates among eligible group members, not shares among selected candidates. It is an adverse-impact indicator, not a universal compliance safe harbour or a rule that every small-sample fluctuation establishes discrimination. See the EEOC Uniform Guidelines questions and answers. Demographic parity, equalised odds, calibration and individual fairness answer different questions. Tradeoff results require stated assumptions; they are not a theorem that every pair of fairness metrics is incompatible.

**Proposed controls to test in the simulator.** Separate a policy's ranking action from human overrides and delayed outcomes. Exercise prompt injection, fabricated feedback, missing group counts, unsupported target actions and exhausted capacity. A shield proves only its encoded invariant under its model assumptions. A regex verifier, feature attribution or Lagrange multiplier alone does not certify a hiring system's fairness, privacy or compliance. Training-data leakage is a threat to investigate, not an established finding about this synthetic repository.

# 10 Phased rollout roadmap

The following phases are a hypothetical planning exercise, not an approved deployment schedule. Durations, sample sizes and numeric gates are illustrative and have not been validated for real hiring. Complete the exercises on synthetic data; any real experiment requires its own review, evidence and decision authority.

**Phase 0 — Instrument (weeks 1–4).** The single highest-leverage activity. Without this, nothing else works.

- Add `policy_version` and `propensity` fields to the decision-event log.
- Introduce an $\varepsilon = 0.02$ uniform exploration layer above the top-$k$ shortlist surface. Recruiters see the same UI; position 1 is sometimes pos-2 from the baseline scorer. No model change yet.
- Stand up the OPE service: a Python job that nightly produces SNIPS / DR estimates for any candidate counterfactual policy.
- **Go/no-go gate**: $\geq 5000$ logged decisions with valid propensities. ESS on the LinTS-argmax target policy $\geq 100$.

**Phase 1 — Shadow-mode bandit (weeks 5–10).** Run LinTS in shadow on every decision. Log the proposed reranking but do not surface it. Nightly OPE estimate vs the heuristic.

- Target metric: SNIPS(LinTS) − SNIPS(heuristic) $\geq 0.03$ absolute hire-rate, with a 95% CI that excludes zero.
- Fairness audit: parity ratio between LinTS proposed and baseline. Must not be worse than the baseline by more than 5 percentage points.
- **Go/no-go**: both above plus recruiter qualitative review of 50 randomly sampled rerank diffs.

**Phase 2 — OPE-validated A/B (weeks 11–16).** Activate LinTS on a single low-risk slot — say, position-3 of the shortlist — on a 10% recruiter cohort.

- Hold-out for OPE comparison.
- Live metric: shortlist → interview conversion rate.
- Fairness monitor: real-time parity ratio with alarm at 0.8.
- Rollback button: instantaneous revert to baseline on any alarm.

**Phase 3 — Constrained online learning (weeks 17–22).** LinTS is replaced by PPO-Lagrangian (D9 also enabled). Constraint on the 4/5 parity rule, plus shield around right-to-work and other compliance gates. Reranking expands to the full top-10.

- Specify whether the requirement is an expected-rate bound, a chance constraint or a per-decision invariant; each needs different evidence.
- Recruiter override is logged but not counted as policy reward signal.
- **Go/no-go**: hire-rate uplift $\geq 5\%$ relative, constraint satisfied, no compliance incidents in 4 weeks.

**Phase 4 — LLM-component DPO (weeks 23–26).** DPO on outreach copy (D5), GRPO on JD polishing (D11), both with length-controlled rewards and verifier-in-the-loop.

- Pairwise recruiter preference collection wired into the compose surface.
- Verifier: legal phrasing regex + length cap + sensitive-term blacklist.
- Metric: outreach reply rate, JD-applicant volume, JD-applicant-quality.

**Phase 5 (later, post-roadmap) — Offline RL on D12.** Once 18+ months of historical offer-acceptance-retention data is available, train IQL offline for salary anchor suggestion. *Never* deploy online; recommendations are presented as a band to the recruiter, who decides.

# 11 Empirical demo summary

The `project/` directory contains a self-contained demo of the Phase-1 reranker plus the constrained variant plus the audit. Running `python -m project.train` on a fresh checkout produces, in under 30 seconds on CPU:

- A 1500-step logged stream from an $\varepsilon$-soft heuristic behaviour policy.
- A trained LinTS reranker, SNIPS and DR estimates for it, the heuristic, and a uniform policy.
- A trained constrained variant whose B/A parity ratio is driven back above the 0.8 floor by the Lagrange multiplier.
- A printed audit report (selection rates, parity ratio, hire rate).

The figures in chapter 6 show all three plots. The workbook ships ten graded exercises that take the reader through each piece of the pipeline.

# 12 Matching application

**Matching application**

This entire week is a matching application: an audit design for a hypothetical recruitment platform, evaluated with synthetic fixtures.

**Headline recommendation.** Start with D1 (candidate re-ranking within a job's shortlist). It is the highest-value, lowest-risk decision in the catalog. The blueprint is week 11's contextual bandit plus week 22's CMDP constraint plus week 23's shield, in that order. Everything else — the LLM DPO, the offline RL on salary, the Decision Transformer on careers — can wait until D1 is shipped and the event-log instrumentation it forces (propensity logging, policy versioning) is in production.

The single biggest takeaway across the whole 24-week curriculum, when pointed at a real system: *the bottleneck is never the algorithm; it is the data infrastructure that lets you evaluate the algorithm honestly.* Spend Phase 0 on logging. The bandit is the easy bit.

# 13 Exercises

**Exercise 13.0.1** ($\star$)**.** Enumerate at least five decision points in the synthetic ATS pipeline that are candidates for RL. For each, name the RL family.

**Exercise 13.0.2** ($\star\star$)**.** Design a reward function for D1. Propose three reward-hacking strategies an adversarial recruiter could use and a mitigation for each.

*Intuition.* Think: how could the policy inflate the reward signal without producing a real hire? Then what evidence would make the hack detectable?

**Exercise 13.0.3** ($\star\star$)**.** Train LinTS on the synthetic ATS environment for $T = 600$ rounds. Report cumulative reward and compare against the heuristic baseline. Use the reference solution in `project/solutions.py` as the ground truth.

**Exercise 13.0.4** ($\star\star$)**.** Bootstrap a 95% confidence interval for the SNIPS estimate of LinTS.

*Intuition.* Resample the log with replacement, recompute SNIPS, take the 2.5% and 97.5% percentiles. Start with 200 for an interactive preview, then increase the resamples and inspect endpoint stability; 200 alone does not establish tail accuracy.

**Exercise 13.0.5** ($\star\star$)**.** Run the PPO-Lagrangian-style trainer with `parity_floor=0.8`. Plot the B/A ratio and the Lagrange multiplier over time. Explain in two sentences what happens when you raise the floor to 0.95.

**Exercise 13.0.6** ($\star\star$)**.** Sketch the propensity-logging schema as a Python TypedDict. Identify the one field whose absence would silently kill OPE.

**Exercise 13.0.7** ($\star\star$)**.** Implement a shielded ranker that drops blocked candidates from the top-$k$. Confirm it raises when fewer than $k$ candidates survive.

**Exercise 13.0.8** ($\star\star\star$)**.** Write down ordinary DPO for D5 and identify where you would add length control. Show that the unmodified DPO loss at $\theta = \theta_{\text{ref}}$ equals $\log 2$ regardless of preference data. An added nonzero margin or length offset changes this initial-value statement. Why is this a good initialisation property?

**Exercise 13.0.9** ($\star\star\star$)**.** Sketch a Decision-Transformer-style architecture for D13 (career trajectory next-step prediction). What return-to-go signal would you use, given that careers are sparse?

**Exercise 13.0.10** ($\star\star\star$ — meaningful objective)**.** End-to-end: run the Phase-1 shadow-mode reranker, produce the audit report, and identify the single biggest fairness risk visible in the metrics. Propose a concrete fix and show, by re-running the constrained variant, that the fix moves the audit metrics in the right direction.

# 14 Further reading

The companion `extended_reading.pdf` links primary sources on counterfactual learning, fair ranking, LinTS, DPO, constraint feedback and official regulatory guidance. Its questions distinguish paper assumptions, synthetic implementations and claims requiring new evidence.